

Exercise: Write the pseudocodes for min, max, predec & succ. in an RBTree.

Divide & Conquer Paradigm.

- Let's revisit this paradigm, with some interesting problems. [Input $\xrightarrow{\text{divide}}$ smaller inputs $\xrightarrow{\text{conquer}}$ go back.]

Problem 1: 1/2-Majority. Input: Array $A[0..n-1]$
(unordered)

Output: x : $\text{freq}(x, A) > n/2$.

Idea 0: $\forall y \in A$, find $\text{freq}(y, A)$. $\Rightarrow O(n^2)$ time.

Idea 1: If we can sort A , $O(n \log n)$ time.

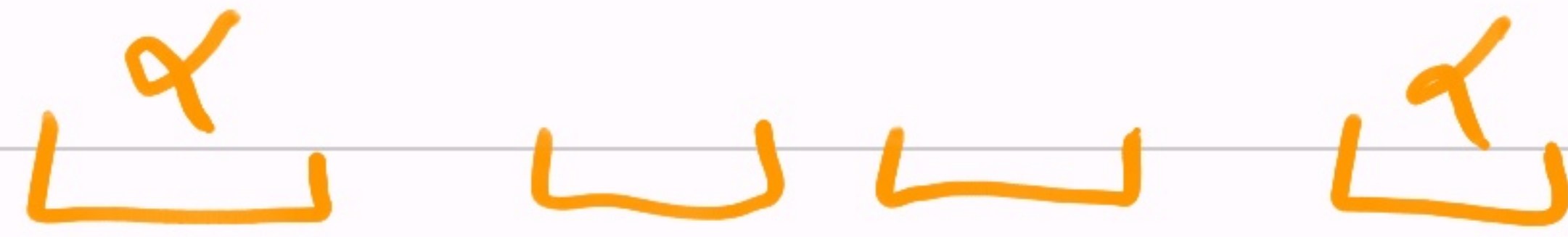
Idea 2: (i) Deleting two unequal elements in A :
e.g. 1. $\alpha \alpha \beta \gamma \alpha \delta \alpha \alpha$ (m pairs assumed)

$\triangleright x$ is $1/2$ -Maj in $A_{\text{old}} \Rightarrow x$ is $1/2$ -Maj in A_{new} .

e.g. 2. $\alpha \gamma \beta \gamma \alpha \delta \alpha$

(ii) When all the pairs are duplicates, reduce

each pair to the respective singleton.
e.g., 1.



▷ (i) & (ii) halve the elements: $|A_{\text{new}}| \leq \frac{1}{2} |A_{\text{old}}|$

⇒ ▷ On repeating this idea, time $\leq$
 $n + n/2 + n/4 + \dots \leq O(n)$.

Qn: Is the output x necessarily $1/2$ -maj of A ?

(iii) Check " $\text{freq}(x, A) > n/2$ ": x is $1/2$ -maj in A

if the check passes, else A has no $\frac{1}{2}$ -Maj.

▷ The above algo. outputs the $\frac{1}{2}$ -Maj, or none, correctly in time $O(n)$.

Qn: This is $O(\lg n)$ scans. Do in 1-scan?
i.e. input is a stream.

Idea 3: keep a count variable, do 1-scan,
keep guessing $\frac{1}{2}$ -Maj x ; & verify.

$\frac{1}{2}$ -Maj ($A[0..n-1]$) {

count $\leftarrow$ 0 ; $x \leftarrow$ nil;

for ($i=0$ to $n-1$) {

if (count = 0) {

$x \leftarrow A[i]$;

count $\leftarrow$ 1;

else if ($x \neq A[i]$) count -- ;

else count ++ ;

} //end for

if ($\text{freq}(x, A) > n/2$) then return x ;

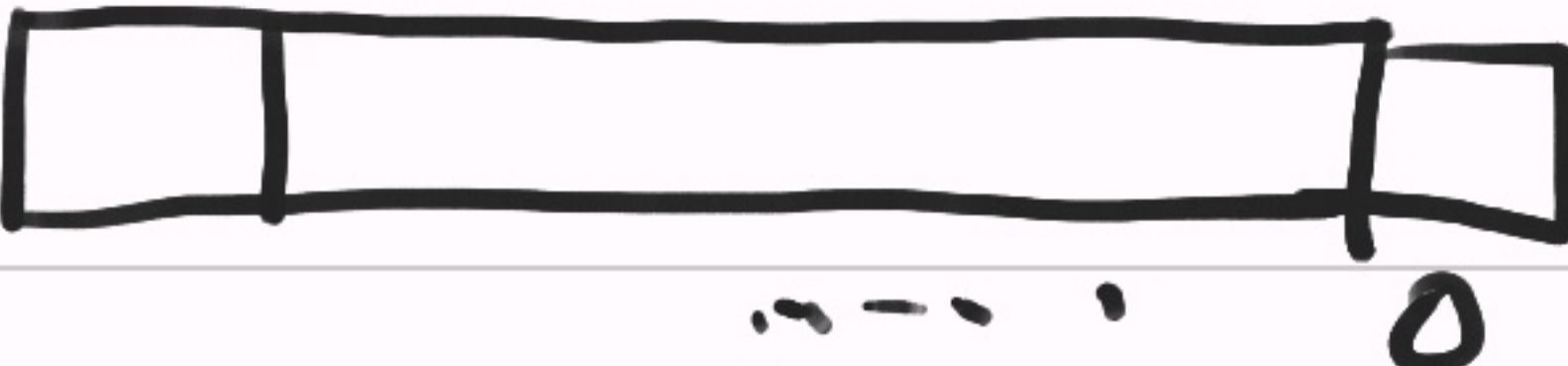
else return nil; }

Theorem: • When it exists, the algo finds $\frac{1}{2}$ -Maj in a stream in $O(n)$ time.
• The algo can verify in an extra scan.
[Boyer-Moore, 1981]

Exercise: Write the invariant & the proof of correctness.

Problem 2: (Integer) Multiplication.

Input: X :  Y : 

Output: $X \cdot Y$. 

Idea 0: School method multiplies $X[i], Y[j]$.
 $\Rightarrow$ time $\geq n^2$.

Idea 1: Divide & Conquer.

Divide the arrays: [Reuse X, Y as the corresponding integer values]

$$X =: \underline{A} \cdot 2^{n/2} + \underline{B}$$

$$Y =: \underline{C} \cdot 2^{n/2} + \underline{D}$$

$$\Rightarrow X \cdot Y = \underline{(AC)} \cdot 2^n + \underline{(AD + BC)} \cdot 2^{n/2} + \underline{(BD)}$$

[Recurrence for time: $T(n) = 4 \cdot T(n/2) + O(n)$
 $\Rightarrow T(n) = O(4^{n/2}) = O(n^2)$.]

Idea 1.1: $(A+B) \cdot (C+D) = \underbrace{(A \cdot C)}_{\checkmark} + \underbrace{(B \cdot D)}_{\checkmark} + \underbrace{(BC+AD)}_{\times}$

$\Rightarrow$ Recurrence now is: $T(n) = 3 \cdot T(n/2) + O(n)$
 $\Rightarrow T(n) = O(3^{\lg n}) = O(n^{\lg 3})$

Theorem [Karatsuba '60]: n -bit integer multiplication takes $O(n^{1.585})$ time.

Reading exercise [Harvey & van der Hoeven 2019]:
 Can be done in $O(n \lg n)$ time.

Problem 3: Inversions. Input: $A[0..n-1]$ of integers, (unsorted)

Output: Count of inversions $:= \# \{(i, j) \mid i < j \ \& \ A[i] > A[j]\}$.
 $D \in [0, \binom{n}{2}]$.

e.g., A :

3	15	8	19	9
---	----	---	----	---

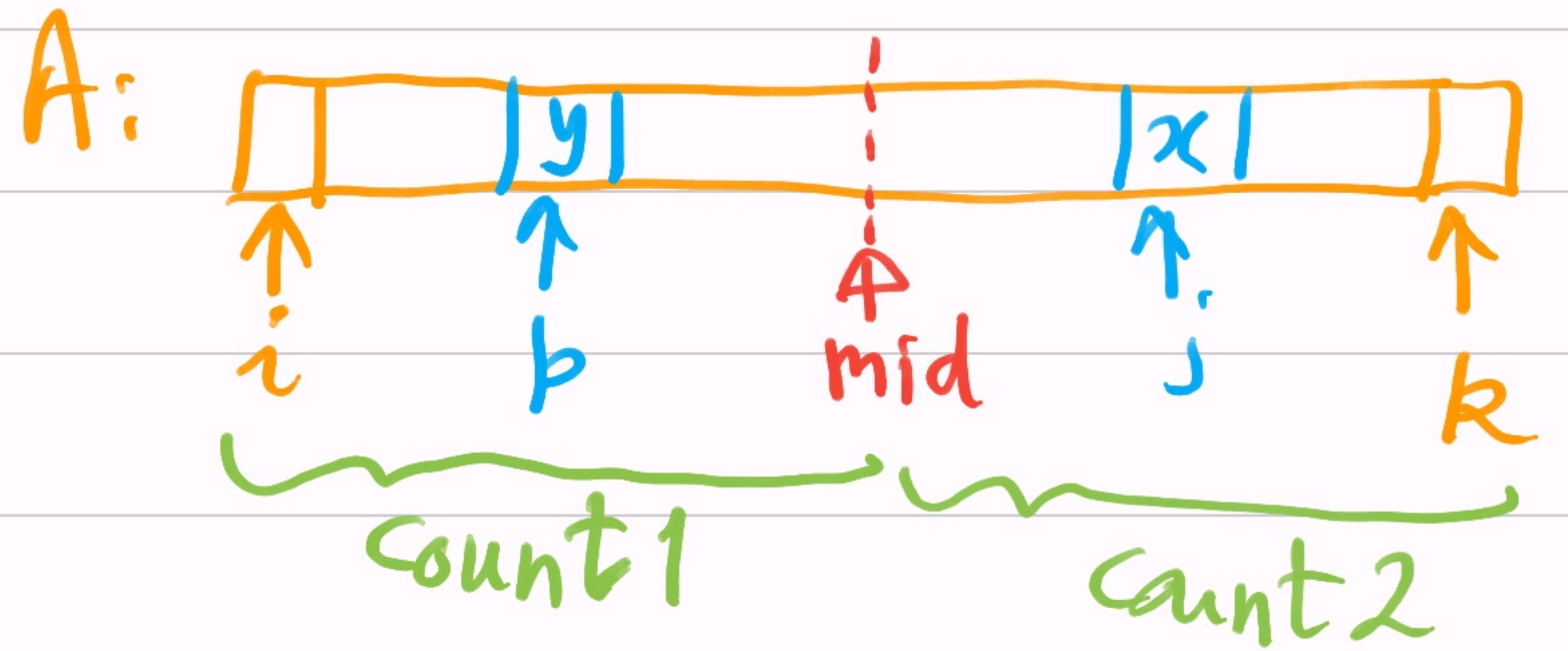
 : Inversions =
 $(1, 2), (1, 4), (3, 4)$.

Idea 0: Double-for loop gives $O(n^2)$ time algo.

Qn: Can we beat n^2 -time?

Idea 1: Use Divide-Conquer! In fact, you can adapt the Merge-Sort algo. a little.

▷ Even if we sort the two halves, count3 is preserved.



- Recursively find $count1$ & $count2$ on the original order.
 - Sort the two halves separately.
 - Qn: During the sorting can we track $count3$?
 - In the merge-algo. " x before y " incident contributes $(mid - p + 1)$ to $count3$.
- $\nwarrow \text{---} \nearrow$
 $count3 = \# \text{ crossover inversions}$

→ This works out to the following pseudocode:

sort & #inv. (A, i, k) }

if (i = k) return 0;

mid $\leftarrow$ (i + k) / 2;

count1 $\leftarrow$ sort & #inv (A, i, mid);

count2 $\leftarrow$ " (A, mid + 1, k);

} $2T(n/2)$

initialize C[0...k-i];

count3 $\leftarrow$ merge & #inv (A, i, mid, k, C); }

copy C into A;

$O(n)$

}

return count1 + count2 + count3;

$$\Rightarrow \triangleright \text{time} = T(n) = 2T(n/2) + O(n).$$

$$\Rightarrow T(n) = O(n \lg n) \text{ for counting inversions.}$$

Exercise: Write the code for merge & #inv(-) .